Order Inventory Application

Application Endpoints

Customer Management APIs

Domain focus: Customer master data (PII-sensitive, finance/insurance aligned)

Endpoints

- **GET** /api/customers
- **GET** /api/customers/{customerId}
- **POST** /api/customers
- **PUT** /api/customers/{customerId}
- **DELETE** /api/customers/{customerId}

Specialized Queries

- **GET** /api/customers/search?email={email}
- **GET** /api/customers/{customerId}/orders
- **GET** /api/customers/{customerId}/shipments

Key Notes

- Enforce **unique email constraint**
- Handle **PII validation**
- Read-only links to Orders & Shipments

Product & Inventory APIs

Domain focus: Product catalog + stock control (core risk in enterprise apps)

Product APIs

- **GET** /api/products
- **GET** /api/products/{productId}
- **POST** /api/products
- **PUT** /api/products/{productId}
- **DELETE** /api/products/{productId}

Inventory APIs

- **GET** /api/inventory
- **GET** /api/inventory/{inventoryId}
- **POST** /api/inventory
- **PUT** /api/inventory/{inventoryId}
- **DELETE** /api/inventory/{inventoryId}

Inventory Filters

- **GET** /api/inventory/store/{storeId}
- **GET** /api/inventory/product/{productId}

Key Notes

- Enforce (store_id, product_id) unique constraint
- Validate stock cannot be negative
- Used downstream by Orders

Store Management APIs

Domain focus: Store/channel master (physical + digital)

Endpoints

- **GET** /api/stores
- **GET** /api/stores/{storeId}
- **POST** /api/stores
- **PUT** /api/stores/{storeId}
- **DELETE** /api/stores/{storeId}

Store-linked Data

- **GET** /api/stores/{storeId}/inventory
- **GET** /api/stores/{storeId}/orders
- **GET** /api/stores/{storeId}/shipments

Key Notes

- Enforce **unique store name**

- Validate **either physical OR web address exists**
- Optional logo upload support

Order Management APIs

Domain focus: Core transactional workflow (financial-grade domain)

Order APIs

- **GET** /api/orders
- **GET** /api/orders/{orderId}
- **POST** /api/orders
- **PUT** /api/orders/{orderId}
- **DELETE** /api/orders/{orderId}

Order Queries

- **GET** /api/orders/customer/{customerId}
- **GET** /api/orders/store/{storeId}
- **GET** /api/orders/status/{status}
- **GET** /api/orders/date-range?from=&to=

Order Status Lifecycle

- **PATCH** /api/orders/{orderId}/status

Allowed status values: OPEN → PAID → SHIPPED → COMPLETE OPEN → CANCELLED PAID → REFUNDED

Key Notes

- Enforce **CHECK constraint on order_status**
- Coordinate with Inventory & Order Items
- Financial transaction integrity

Order Items & Shipment APIs

Domain focus: Fulfillment, logistics & line-level pricing

Order Items APIs

- **GET** /api/orders/{orderId}/items
- **POST** /api/orders/{orderId}/items
- **PUT** /api/orders/{orderId}/items/{lineItemId}
- **DELETE** /api/orders/{orderId}/items/{lineItemId}

Item Constraints

- Unique (order_id, product_id)
- Quantity > 0
- Unit price immutable after shipment

Shipment APIs

- **GET** /api/shipments
- **GET** /api/shipments/{shipmentId}
- **POST** /api/shipments
- **PUT** /api/shipments/{shipmentId}
- **DELETE** /api/shipments/{shipmentId}

Shipment Filters

- **GET** /api/shipments/customer/{customerId}
- **GET** /api/shipments/store/{storeId}
- **GET** /api/shipments/status/{status}

Shipment states: CREATED → SHIPPED → IN-TRANSIT → DELIVERED

Key Notes

- Shipment creation links to Orders & Order Items
- Status transitions are controlled operations

High-Impact Reporting Endpoints

✔ **1. Sales Summary Report**

Endpoint
HTTP
GET /api/reports/sales/summary?from=&to=&storeId=

What it should return
- Total Revenue
- Total Orders
- Average Order Value

Why this is powerful
- Requires joins: orders + order_items
- Uses aggregation: SUM, COUNT
- Uses filters: date range, store
- Must consider only valid business states like COMPLETE orders

What you evaluate
- SQL correctness (JOIN + GROUP BY)
- Understanding of revenue calculation
- Ability to interpret business data

✔ 2. Top Customers Report

Endpoint
HTTP
GET /api/reports/customers/top?limit=10

What it should return
- Customer Name
- Total Orders
- Total Spend

Why this is powerful
- Requires 3-table join: customers + orders + order_items
- Uses grouping by customer
- Applies business concept: customer value

What you evaluate
- Handling foreign keys & relationships
- Correct aggregation logic
- Ability to rank & sort (TOP N)

✔ 3. Inventory Risk Report

Endpoint
HTTP
GET /api/reports/inventory/risk

What it should return
- Product ID
- Current Stock
- Total Sold
- Risk Level (LOW / CRITICAL)

Logic
- Stock from inventory table
- Sales from order_items
- Example rule:
  - stock < 5 → CRITICAL
  - stock < sales → RISK

Why this is powerful
- Real-world scenario: demand vs supply

Order Inventory Application

- Requires multi-table join + business logic layer
- Not just SQL → requires thinking

What you evaluate

- Ability to combine analytics + business rules
- Problem-solving beyond CRUD APIs
- Understanding of inventory management